

Least Squares; Sequence Alignment

1 Segmented Least Squares

- multi-way choices
- applying dynamic programming

2 Sequence Alignment

- matching similar words
- applying dynamic programming
- analysis of the algorithm

CS 401/MCS 401 Lecture 13
Computer Algorithms I
Jan Verschelde, 17 July 2026

Least Squares; Sequence Alignment

1 Segmented Least Squares

- multi-way choices
- applying dynamic programming

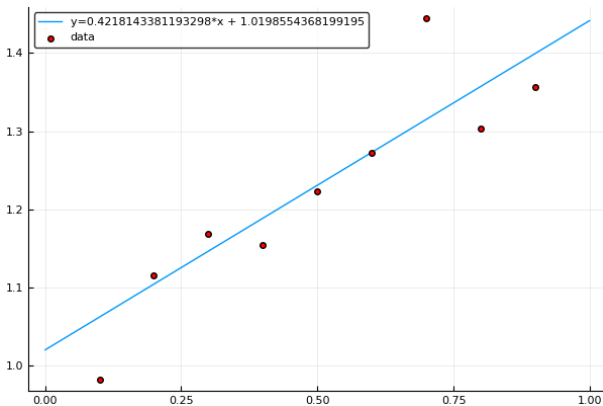
2 Sequence Alignment

- matching similar words
- applying dynamic programming
- analysis of the algorithm

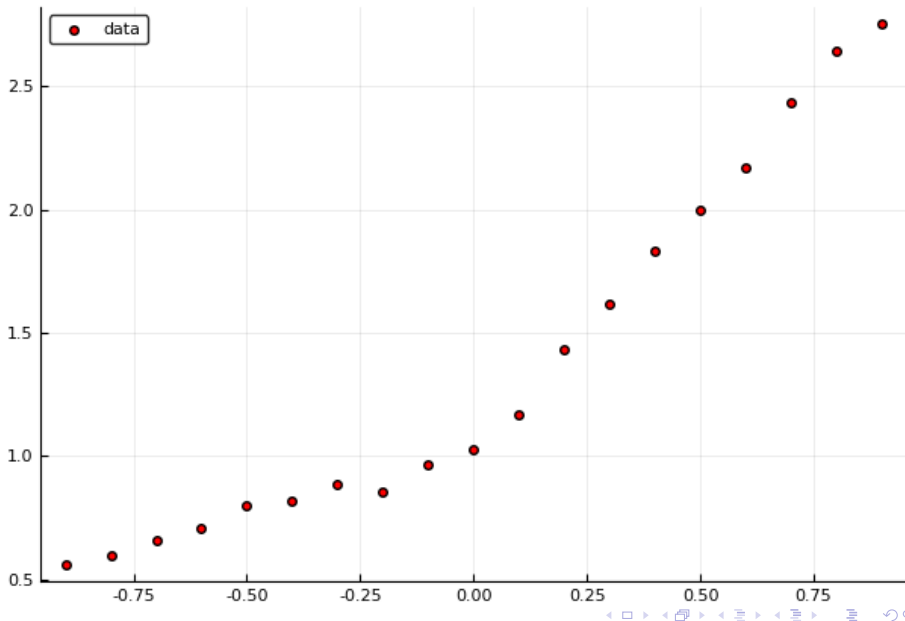
fitting data in the least squares sense

Input: n points (x_i, y_i) , $x_1 < x_2 < \dots < x_n$.

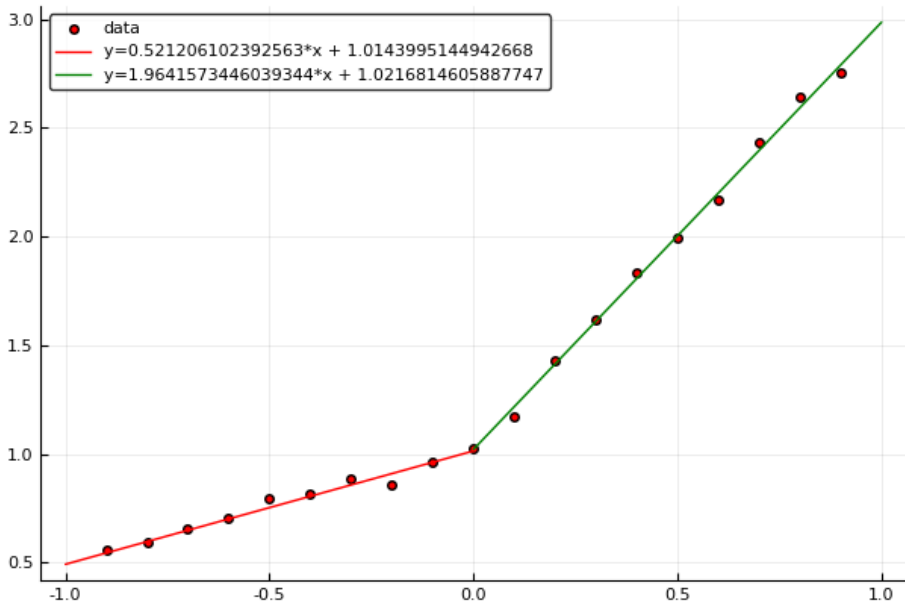
Output: a, b , coefficients of $y = ax + b$, so $\sum_{i=1}^n (y_i - ax_i - b)^2$ is minimal over all a and b .



data close to two lines



fitting data close to two lines



explicit formulas for a linear least squares fit

The coefficients a and b of the line $y = ax + b$ for the least squares fit of the n points (x_i, y_i) satisfy

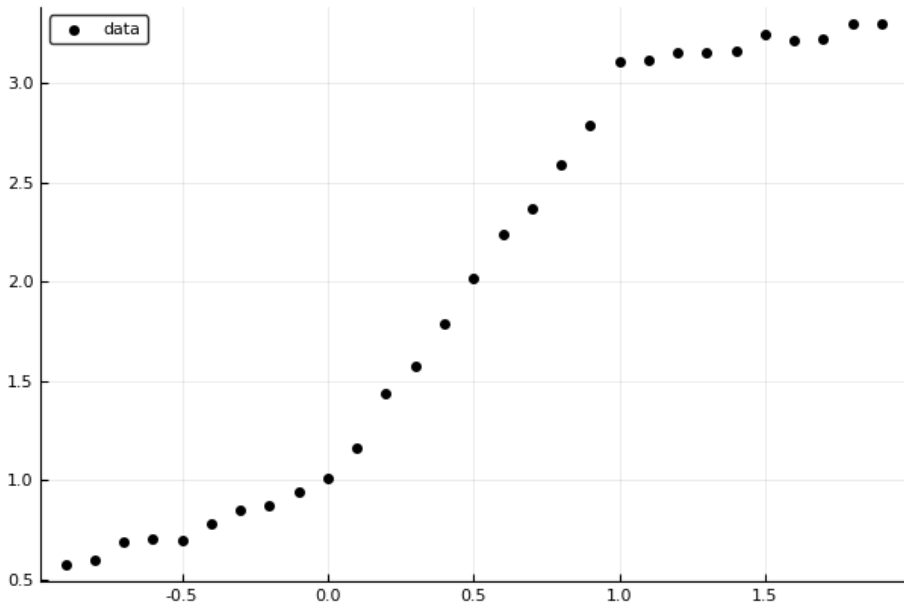
$$a = \frac{n \sum_{i=1}^n x_i y_i - \left(\sum_{i=1}^n x_i \right) \left(\sum_{i=1}^n y_i \right)}{n \sum_{i=1}^n x_i^2 - \left(\sum_{i=1}^n x_i \right)^2}$$

$$\text{and } b = \frac{1}{n} \left(\sum_{i=1}^n y_i - a \sum_{i=1}^n x_i \right).$$

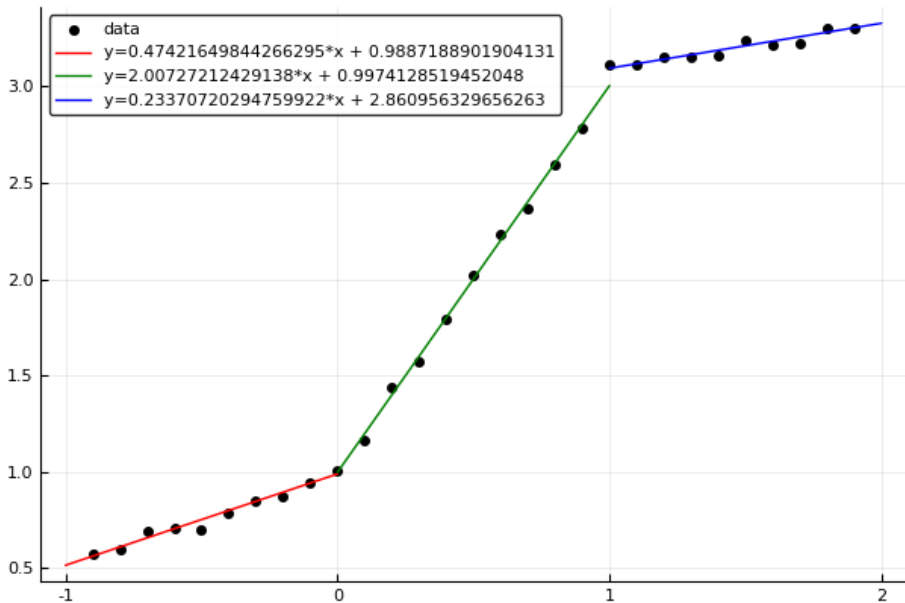
Exercise 1:

Express the cost to compute a and b as $O(f(n))$, for some function f . Justify your formula for $f(n)$.

problem: detect the direction of change



solved by three linear least squares fits



data points and segments

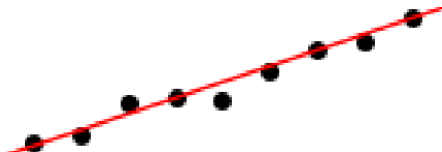
Input: n points (x_i, y_i) , $x_1 < x_2 < \dots < x_n$.

Each point is represented by an index i , $i \in \{1, 2, \dots, n\}$.

Definition (1)

Given a set of n indices $\{1, 2, \dots, n\}$, **a segment (i, j)** , with $i \leq j$, defines the sequence of points $i, i+1, \dots, j$.

A segment defined by the first nine points of the data:



partition the data into segments

Input: n points (x_i, y_i) , $x_1 < x_2 < \dots < x_n$.

The data will be segmented into a partition.

Definition (2)

Given a set of n indices $\{1, 2, \dots, n\}$, *a partition into segments* is a sequence of m segments (i_k, j_k) , for $k = 1, 2, \dots, m$, with

- $i_1 = 1$ and $j_m = n$;
- $i_{k+1} = j_k + 1$, for $k = 1, 2, \dots, m - 1$.

What is the best way to partition the data?

the segmented least squares problem

Input: n points (x_i, y_i) , $x_1 < x_2 < \dots < x_n$.

Output: a partition of the points in segments with minimal penalty.

Definition (3)

Given n points (x_i, y_i) , $x_1 < x_2 < \dots < x_n$ and a partition into m segments (i_k, j_k) , **the penalty** is the sum of

1 $m \times C$, where $C > 0$;

2 $\sum_{\ell=i_k}^{j_k} (y_\ell - ax_\ell - b)^2$ is the sum of squared distances of the linear fit to the data points in the k -th segment.

Is there a unique solution?

Exercise 2:

Does the segmented least squares problem have a unique solution?

Justify your answer with proof and/or examples.

Least Squares; Sequence Alignment

1 Segmented Least Squares

- multi-way choices
- applying dynamic programming

2 Sequence Alignment

- matching similar words
- applying dynamic programming
- analysis of the algorithm

guidelines for dynamic programming

For dynamic programming to apply, we need a collection of subproblems derived from the original problem that satisfies the properties:

- 1 There are a polynomial number of subproblems.
- 2 The solution to the original problem can be easily computed from the solutions to the subproblems.
- 3 There is a natural order from smallest to largest subproblem, jointly with an easy-to-compute recurrence.

In the segmented least squares problem, we develop not a binary, but a *multi-way choices* recurrence: at each step we have a polynomial number of possibilities to consider for the structure of the optimal solution.

a useful observation

The last segment has to end in n , so $j_m = n$.

What about the start of the last segment? What is i_m ?

The penalty is the sum of

- 1 C , from the segment (i_m, n) ,
- 2 $\sum_{\ell=i_m}^n (y_\ell - a_m x_\ell - b_m)^2$, the error of the fit, and
- 3 the penalty of the partition of $\{1, 2, \dots, i_m - 1\}$.

Observe that the value of m does not matter. Abbreviate i_m by i .

finding a good notation for the observation

For $i \in \{1, 2, \dots, n\}$ denote by $\text{OPT}(i)$ the penalty for the partition of $\{1, 2, \dots, i\}$ in an optimal solution. In the trivial case, $\text{OPT}(0) = 0$.

Let $e_{i,n} = \sum_{\ell=i}^n (y_\ell - a_m x_\ell - b_m)^2$ denote the error of the fit.

The useful observation translates then into a recurrence.

Lemma (4)

Let the last segment in an optimal partition of n data points begin at i . Then, the value of that optimal solution is

$$\text{OPT}(n) = C + e_{i,n} + \text{OPT}(i - 1).$$

designing a general recurrence

The previous recurrence has its focus on the last segment. We still have not answered the *What is i ?* question.

What if we tried all values for i , for $i = 1, 2, \dots, n$?
For every tried value for i , consider:

- taking i as the start of a segment has the cost $C + e_{i,n} + \text{OPT}(i - 1)$,
- optimizing happens by taking the minimum cost over all i .

This leads then to a more general recurrence:

Lemma (5)

The value of an optimal solution of a partition of n points into segments is

$$\text{OPT}(n) = \min_{1 \leq i \leq n} (C + e_{i,n} + \text{OPT}(i - 1)).$$

a recursive solution

Exercise 3:

The recurrence

$$\text{OPT}(n) = \min_{1 \leq i \leq n} (C + e_{i,n} + \text{OPT}(i - 1))$$

suggests a recursive algorithm.

- 1 Formulate the recursive algorithm in pseudo code.
- 2 For $n = 5$, draw the tree of subproblems.
- 3 Extrapolate your drawing of the tree of subproblems for $n = 5$ into a general formula for the cost of the recursive algorithm, for any n .

an iterative algorithm

Segmented-Least-Squares($n, \mathbf{x}, \mathbf{y}, C$)

Input: n , the number of data points,

$\mathbf{x}$, x -coordinates $x_1 < x_2 < \dots < x_n$,

$\mathbf{y}$, y -coordinates of the data points,

C , the penalty for a segment.

Output: returns $\text{OPT}(n)$.

let M be an array indexed by $0, 1, \dots, n$

$M[0] := 0$

for all pairs (i, j) , $i \leq j$, do $e_{i,j} := \sum_{\ell=i}^j (y_\ell - a_{i,j}x_\ell - b_{i,j})^2$

for $j = 1, 2, \dots, n$ do

$M[j] := \min_{1 \leq i \leq j} (C + e_{i,j} + M[i - 1])$

return $M[n]$

proving the correctness

Exercise 4:

Prove that `Segmented-Least-Squares` is correct.

- 1 Formulate a precise statement on the correctness.
- 2 Use the recurrence relation in your proof.

the cost of Segmented-Least-Squares

Theorem (6)

For n points, Segmented-Least-Squares runs in $O(n^3)$ time.

Proof. The computation of $e_{i,j}$, for all pairs (i,j) , for $1 \leq i \leq j \leq n$ requires $O(n^2)$ steps. By exercise 1, each step requires $O(n)$ operations to compute the coefficients $a_{i,j}$ and $b_{i,j}$. The first for loop in Segmented-Least-Squares thus runs in $O(n^3)$ time.

The second for loop does n steps, for all $j = 1, 2, \dots, n$.

The operation in the loop $\min_{1 \leq i \leq j}$ requires

- j sums of three elements to compute $C + e_{i,j} + M[i - 1]$,
- j comparisons of the current minimum with $C + e_{i,j} + M[i - 1]$.

So the second loop runs in $O(n^2)$ time.

As $O(n^3) + O(n^2)$ is $O(n^3)$,

Segmented-Least-Squares runs in $O(n^3)$ time.

Q.E.D.

reporting segments in an optimal solution

`Report-Segments( $n, C, e_{i,j}, M$ )`

Input: n , the number of data points,
 C , the penalty for a segment,
 $e_{i,j}$, fitting errors for all segments (i, j) ,
 M , the array in `Segmented-Least-Squares`.

Output: prints all segments in the optimal partition.

if $n = 0$ then

 print nothing

else

 find i that minimizes $C + e_{i,j} + M[i - 1]$

 print (i, n)

`Report-Segments( $i - 1, C, e_{i,j}, M$ )`

Do we have an efficient algorithm?

Exercise 5:

- What is the cost of `Report-Segments`? Give a formula $f(n)$ for the cost, in function of the number n of data points.
- Justify your $O(f(n))$ formula.

Least Squares; Sequence Alignment

1 Segmented Least Squares

- multi-way choices
- applying dynamic programming

2 Sequence Alignment

- matching similar words
- applying dynamic programming
- analysis of the algorithm

CS 401/MCS 401 Lecture 13
Computer Algorithms I
Jan Verschelde, 17 July 2026

Least Squares; Sequence Alignment

1 Segmented Least Squares

- multi-way choices
- applying dynamic programming

2 Sequence Alignment

- **matching similar words**
- applying dynamic programming
- analysis of the algorithm

matching similar words

Consider the following situations.

- Search an online dictionary for a word.

Often a user may be so unfamiliar with a word that the exact spelling of the word is not known.

- Compile a program.

A helpful parser will match misspelled keywords or variable names and offer suggestions to correct the errors.

Words for us will be sequences of characters $x \in \{ 'a', 'b', \dots, 'z' \}$.

mismatched characters

Our alphabet is $\mathcal{A} = \{ 'a', 'b', \dots, 'z' \}$.

As an example of a mismatch,
consider $X = \text{occurrence}$ and $Y = \text{occurrence}$.

There is a mismatch at positions 6 and 6 of both x and y ,
as $X_6 = a$ and $Y_6 = e$.

Definition (7)

Let X and Y be two words of lengths respectively m and n :
 $X = x_1x_2 \cdots x_m$ and $Y = y_1y_2 \cdots y_n$. A **mismatch** is a pair (i, j) ,
 $i \in \{ 1, 2, \dots, m \}$ and $j \in \{ 1, 2, \dots, n \}$, for which $x_i \neq y_j$.

missing characters

Misspelled words may have missing characters.

As an example of a gap,
consider $X = \text{o-currence}$ and $Y = \text{occurrence}$.

There is a gap at positions 2 and 2 of both x and y ,
as $x_2 = -$ and $y_2 = c$.

Definition (8)

Let X and Y be two words of lengths respectively m and n :
 $X = x_1x_2 \cdots x_m$ and $Y = y_1y_2 \cdots y_n$. A **gap** is a pair (i, j) ,
 $i \in \{1, 2, \dots, m\}$ and $j \in \{1, 2, \dots, n\}$,
for which $x_i = -$ and $y_j \neq -$ or $x_i \neq -$ and $y_j = -$.

matching two words

In the definition below we consider two words X and Y , of lengths respectively m and n : $X = x_1x_2 \cdots x_m$ and $Y = y_1y_2 \cdots y_n$.

Definition (9)

A **matching** is a sequence of pairs (i, j) , $i \in \{1, 2, \dots, m\}$ and $j \in \{1, 2, \dots, n\}$, such that each x_i and y_j occurs in at most one pair.

Observe that the matching is defined by the indices, not the characters in the words.

Consider for example, $X = \text{solution}$ and $Y = \text{book}$.

- $(2, 2)$ and $(7, 3)$ is a matching, but
- $(2, 2)$ and $(2, 3)$ is not.

an alignment between two words

In the definition below we consider two words X and Y , of lengths respectively m and n : $X = x_1x_2 \cdots x_m$ and $Y = y_1y_2 \cdots y_n$.

We are interested in computing alignments between words.

Definition (10)

A matching M is *an alignment* if there are no crossing pairs:
for all $(i, j), (k, \ell) \in M$: if $i < k$, then $j < \ell$.

For example, for $x = \text{stop-}$ and $y = \text{-tops}$,
the alignment is $(2, 1), (3, 2), (4, 3)$.

problem: find an optimal alignment

Definition (11)

Suppose M is a given alignment between X and Y :

- 1 The **gap cost** δ is applied for each pair $(i, j) \in M$ which is a gap.
- 2 The **mismatch cost** α_{x_i, y_j} is applied for each pair $(i, j) \in M$ which is a mismatch.

The **cost of M** is the sum of all gap and mismatch costs.

Note that the parameters determine which alignment of “ocurrance” and “occurrence” we should prefer:

- o—currance and occurence have a cost of $\delta + \alpha_{a,e}$,
- o—curr—ance and occure—nce have a cost of 3δ .

longest common subsequence

Given two sequences,
what is the length of the longest common subsequence?

A sequence $z_1, z_2, \dots, z_k$ is a subsequence of x
if there is a strictly increasing sequence of indices $i_1 < i_2 < \dots < i_k$
such that $x_{i_j} = z_j$, for $j = 1, 2, \dots, k$.

Example: b, c, d, b is a subsequence of a, b, c, b, d, a, b
for indices 2, 3, 5, 7.

For given sequences x and y , the longest common subsequence

- either ends at x_i and y_j ,
- or ends at x_i and y_{j-1} ,
- or ends at x_{i-1} and y_j ,

where i and j are valid indices in x and y .

Exercise 6: Describe the similarities and differences between the longest common subsequence and sequence alignment.

Least Squares; Sequence Alignment

1 Segmented Least Squares

- multi-way choices
- applying dynamic programming

2 Sequence Alignment

- matching similar words
- **applying dynamic programming**
- analysis of the algorithm

a dichotomy

Let X and Y be two words of length m and n respectively.

In an optimal solution M ,

- either $(m, n) \in M$,
- or $(m, n) \notin M$.

We need a stronger statement.

Lemma (12)

Let M be any alignment of X and Y . If $(m, n) \notin M$, then

- *either the m -th position of X is not matched in M ,*
- *or the n -th position of Y is not matched in M .*

the lemma with proof

Lemma (12)

Let M be any alignment of X and Y . If $(m, n) \notin M$, then

- either the m -th position of X is not matched in M ,
- or the n -th position of Y is not matched in M .

Proof. Assume by contradiction there is a matching of the m -th position of X and the n -th position of Y : we have $i < m$ and $j < n$ so that

- $(m, j) \in M$,
- $(i, n) \in M$;

and $(m, n) \notin M$.

Recall that in an alignment there is no crossing, but we have $(i, n), (m, j) \in M$ with $i < m$ and $n > j$, a crossing.

This crossing contradicts the assumption $(m, n) \notin M$.

Q.E.D.

restating the lemma

Lemma (12)

Let M be any alignment of X and Y . If $(m, n) \notin M$, then

- either the m -th position of X is not matched in M ,*
- or the n -th position of Y is not matched in M .*

In our way to a recurrence, we restate the lemma:

Lemma (13)

In an optimal alignment M , at least one of the following holds:

- ① $(m, n) \in M$; or*
- ② the m -th position of X is not matched; or*
- ③ the n -th position of Y is not matched.*

deriving a recurrence

Let $\text{OPT}(i, j)$ denotes the minimum cost of an alignment between

- $x_1 x_2 \cdots x_i$, and
- $y_1 y_2 \cdots y_j$.

The restated lemma has three cases:

- 1 If $(m, n) \in M$, then

$$\text{OPT}(m, n) = \alpha_{x_m, y_n} + \text{OPT}(m - 1, n - 1).$$

- 2 If the m -th position of X is not matched, we have a gap, and then

$$\text{OPT}(m, n) = \delta + \text{OPT}(m - 1, n).$$

- 3 If the n -th position of Y is not matched, we have a gap, and then

$$\text{OPT}(m, n) = \delta + \text{OPT}(m, n - 1).$$

the recurrence

Theorem (14)

The minimum alignment costs satisfy the following recurrence, for $i \geq 1$ and $j \geq 1$:

$$\text{OPT}(i, j) = \min \begin{pmatrix} \alpha_{x_i, y_j} + \text{OPT}(i-1, j-1), \\ \delta + \text{OPT}(i-1, j), \\ \delta + \text{OPT}(i, j-1) \end{pmatrix}.$$

The trivial cases are

- $\text{OPT}(i, 0) = i\delta$, for $i = 1, 2, \dots, m$,
- $\text{OPT}(0, j) = j\delta$, for $j = 1, 2, \dots, n$,

as the only way to line up an k -letter word with a 0-letter word is to use k gaps.

With the recurrence we can formulate an algorithm.

the algorithm

Alignment($m, n, X, Y, \delta, \alpha_{a,b}$)

Input: m and n , the number of letters in X and Y ,
 X and Y , words of length m and n ,
 δ , the gap cost,
 $\alpha_{a,b}$, the mismatch cost between a and b .

Output: returns $\text{OPT}(m, n)$.

let $A[0 \dots m, 0 \dots n]$ be an array of arrays

for i from 0 to m do $A[i, 0] := i\delta$

for j from 0 to n do $A[0, j] := j\delta$

for j from 1 to n do

for i from 1 to m do

$$A[i, j] = \min \begin{pmatrix} \alpha_{x_i, y_j} + A[i-1, j-1], \\ \delta + A[i-1, j], \\ \delta + A[i, j-1] \end{pmatrix}$$

return $A[m, n]$.

Least Squares; Sequence Alignment

1 Segmented Least Squares

- multi-way choices
- applying dynamic programming

2 Sequence Alignment

- matching similar words
- applying dynamic programming
- analysis of the algorithm

the cost of the algorithm

Theorem (15)

For words of length m and n , `Alignment` runs in $O(mn)$ time.

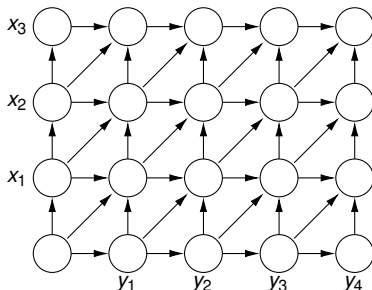
Exercise 7:

Prove the theorem about the cost of `Alignment`.

sequence alignment picture by a graph

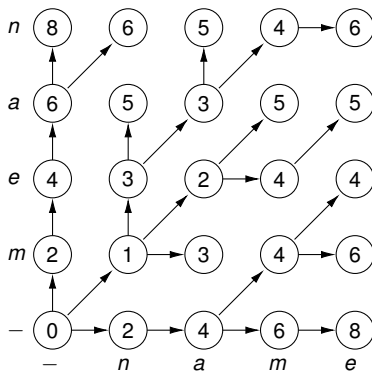
In the sequence alignment problem, we build a graph G_{XY} :

- the nodes are labeled by (i, j) ,
for $i = 0, 1, \dots, m$, and for $j = 0, 1, \dots, n$,
- the weights on the edges are the costs:
 - ▶ the cost of each vertical and horizontal edge is δ ,
 - ▶ the cost of each diagonal edge from $(i-1, j-1)$ to (i, j) is α_{x_i, y_j} .



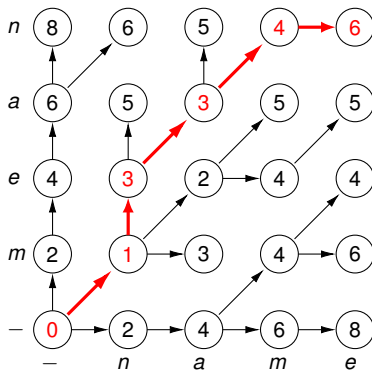
the OPT values for aligning *mean* to *name*

- 1 the cost of a gap is $\delta = 2$,
- 2 matching a vowel with a different vowel costs 1,
- 3 matching a consonant with a different consonant costs 1,
- 4 matching a vowel with a different consonant costs 3.



the minimum cost path in G_{XY}

In the OPT values for aligning *mean* to *name*, follow the arrows backward to construct the alignment



The minimum cost is 6, aligns $\begin{matrix} m & e & a & n & - \\ n & - & a & m & e \end{matrix}$.

the minimum cost path

Proposition (16)

Let $f(i, j)$ denote the minimum cost of a path from $(0, 0)$ to (i, j) in G_{XY} . Then, for all i, j , we have $f(i, j) = \text{OPT}(i, j)$.

Proof. We apply induction on $i + j$.

Base case: if $i + j = 0$, then $i = j = 0$, and $f(0, 0) = \text{OPT}(0, 0) = 0$.

Induction hypothesis: assume true for all (i', j') with $i' + j' < i + j$.

We must show $f(i, j) = \text{OPT}(i, j)$.

The last edge on the shortest path to (i, j) is either

- from $(i - 1, j - 1)$,
- from $(i - 1, j)$, or
- from $(i, j - 1)$.

proof continues

The last edge on the shortest path to (i, j) is either from $(i - 1, j - 1)$, from $(i - 1, j)$, or from $(i, j - 1)$.

$$f(i, j) = \min \left(\alpha_{x_i y_j} + f(i - 1, j - 1), \delta + f(i - 1, j), \delta + f(i, j - 1) \right)$$

Then we can apply the induction hypothesis, as $i - 1 + j - 1 < i + j$, $i - 1 + j < i + j$, and $i + j - 1 < i + j$, so $f(i, j)$ equals

$$\min \left(\alpha_{x_i y_j} + \text{OPT}(i - 1, j - 1), \delta + \text{OPT}(i - 1, j), \delta + \text{OPT}(i, j - 1) \right)$$

and, by Theorem (13), we obtain $\text{OPT}(i, j)$.

Q.E.D.

the length of the longest common subsequence

Let $c_{i,j}$ be the length of a common subsequence of $x_1 \dots x_i$ and $y_1 \dots y_j$, then

$$c_{i,j} = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ 1 + c_{i-1,j-1} & \text{if } i > 0, j > 0 \text{ and } x_i = y_j \\ \max(c_{i,j-1}, c_{i-1,j}) & \text{if } i > 0, j > 0 \text{ and } x_i \neq y_j \end{cases}$$

Exercise 8:

Given computed values $c_{i,j}$,
define an algorithm to compute the longest common subsequence.